

## Week 2 - Meeting (2/20) Recap:

- In today's meeting, we confirmed our solution with our mentors and aligned on creating an **AI-Agent-based Product Risk Management System**.

### *We confirmed that:*

- **Natural Language Processing (NLP)** is essential to automatically categorize review themes, detect complaint severity, and standardize interpretations across languages.
- We should create a **document** that has all the **requirements for the Case Competition** so that, regardless of how technical we get, we stay grounded to the main idea of the project.
- The most unique element of our solution is the **Revenue-Weighted Risk Engine**, which allows us to convert the impact of our dashboard to a financial/business performance metric.
- As **Mrs. Edmonds** stated, every technology choice must tie directly to a business outcome, and the focus should remain on delivering measurable improvements in *revenue protection, customer satisfaction, and brand protection*.

### *We came into the meeting with a doubt about "how to make our solutions unique," and we left with:*

- Understanding that implementing the **Product Risk Score** would make our solution compelling, as it connects AI directly to Business impact.
- Adding a dataset extension by **integrating support tickets** allows us to make the system more dynamic with real service data and create a more "live" intelligence model.

## Week 3 Goals / Progress:

- **Set up GitHub Repository (Done)**
    - Updated ReadMe, Add Official Documents, Challenge #1 Pitch Slideshow, and Personalized Case Competition Goals Document
  - **Design Flow of Data (Done)**
    - From entering the database, how does data flow to reach the final “business” action/suggestion?
  - **Evaluate & Finalize the Dataset (Done)**
    - Review time trends, check missing key fields, remove duplicates, and keep only stable years to create a clean, consistent dataset with minimal data loss (~6.4%) ready for reliable modeling.
  - **Set up a centralized SQL database (Done)**
    - Store the dataset in a server, so that data is not refreshed every run, and we reduce redundant computation while improving processing efficiency and system performance.
    - Make sure all the data is accessible and easily updated through a Python Project. Test flow like **pull, modify, push**.
    - Demonstrate how to run the code with the ReadMe for the team members.
  - **Determine How to Validate the Model (Done)**
    - We performed a chronological 90/10 temporal split, using the earlier 90% of reviews for model training and dashboard development, and reserving the most recent 10% for forward validation to simulate real-world scenarios and demonstrate system credibility.
    - Plan how to generate synthetic support tickets and reviews over a multi-month span to simulate both no-trend and small long-term trend scenarios, assessing whether the model can correctly report no new risk or detect emerging risk early enough to estimate impact (e.g., revenue saved).
    - Put into words, the idea of replaying real past data to see if the model would have warned earlier about a drop in ratings and estimate how much revenue could have been saved.
- 

## Upcoming Week Tasks:

- **Landon:** Design UI / UX for Model Interface
- **Sathvik:** Create Synthetic Support Tickets (**Make sure to refer to Dell Technologies’ Community Forum**)
- **Mayank:** Upload Synthetic Support Tickets + Upload *Replay* Dataset
- **Daniel and Makena:** Design Model and Start Coding!